MLOps on Azure Machine Learning — Pipelines, Registry & Monitoring

Implementation Solution Guide

Version 1.0.0 · 2026-07-10

Contents

Executive Summary — MLOps on Azure Machine Learning 4

What you get . . . 4

What this delivers . . . 4

The standard deployment model . . . 4

Outcome . . . 5

1. Solution Overview — MLOps on Azure Machine Learning 5

What MLOps on Azure ML is . . . 5

How the technology works . . . 5

What the MLOps foundation contains . . . 6

What you can control . . . 6

What is out of scope / limitations (be honest with buyers) . . . 7

Authentication and connections used . . . 7

The standard deployment model . . . 7

2. Requirements — MLOps on Azure Machine Learning 8

R1. Azure ML CLI extension and `azure-ai-ml` SDK v2 at supported versions . . . 8

R2. Rights to create the Azure ML workspace and its linked resources . . . 8

R3. Non-production Azure ML workspace to pilot in . . . 8

R4. A registered data asset (training data) with a known permission model . . . 8

R5. Managed identity and least-privilege data-plane roles for compute and endpoints . . . 9

R6. Model evaluation metric and acceptance threshold defined (by the data-science owner) . . . 9

R7. Pipeline approval gate and reviewer group for register/deploy . . . 9

R8. Monitoring baseline dataset and an alert action group . . . 9

Global Requirements — AI (category-level) 10

G1. Azure subscription with AI service access and rights . . . 10

G2. Entra identity, managed identity, and RBAC . . . 10

G3. Private networking capability . . . 10

G4. IaC + Python tooling and source control . . . 11

G5. CI/CD platform . . . 11

G6. Data source and classification (for data-grounded solutions) . . . 11

G7. Content-safety and responsible-AI posture . . . 11

G8. Cost controls and quotas . . . 11

3. Implementation — MLOps on Azure Machine Learning 11

Naming you will create (reuse these exact names) . . . 12

Step 1 — Assess: inventory the current ML practice (read-only) . . . 12

Step 2 — Configure: provision the workspace and compute as code (non-prod first) . . . 13

Step 3 — Configure: register the data asset and define the environment as code . . . 13
Step 4 — Configure: run the reproducible train + evaluate pipeline (registration in DRY-RUN) . . . 14
Step 5 — Configure: register the candidate model (gated on the metric) . . . 14
Step 6 — Pilot: deploy to a managed online endpoint (blue/green) . . . 14
Step 7 — Wire the CI/CD pipeline (train → register → approval → deploy) . . . 15
Step 8 — Configure: turn on drift and performance monitoring with alerts . . . 15
Step 9 — Validate, then Expand: production workspace behind approval . . . 15
5. Testing — MLOps on Azure Machine Learning 16
Test 1 — Workspace and compute exist as code . . . 16
Test 2 — A training run is reproducible and tracked . . . 16
Test 3 — Registration is gated on the metric threshold . . . 16
Test 4 — Model registry versioning and lineage . . . 17
Test 5 — Managed online endpoint scores . . . 17
Test 6 — Blue/green rollback works . . . 17
Test 7 — Drift/performance monitors fire and alert . . . 17
Test 8 — Pipeline enforces the metric gate and approval . . . 17
Exit criteria . . . 18
6. Troubleshooting — MLOps on Azure Machine Learning 18
Tooling / SDK . . . 18
Workspace / infrastructure deploy . . . 18
Training run / reproducibility . . . 18
Registration / metric gate . . . 19
Model promotion . . . 19
Endpoints . . . 19
Monitoring . . . 19
CI/CD pipeline . . . 20
7. Best Practices — MLOps on Azure Machine Learning 20
Keep it clean . . . 20
Decisions that belong to the customer . . . 21
Where to go next . . . 21
8. Deliverables Summary — MLOps on Azure Machine Learning 21
Documentation . . . 21
Automation artifacts . . . 22
What the buyer receives . . . 23
Outcome . . . 23
Appendix A — PowerShell Scripts 24

Executive Summary — MLOps on Azure Machine Learning

White-label · AI

Most machine-learning work starts the same way: a data scientist trains a model in a notebook, exports a `.pkl` file, and someone copies it onto a server by hand. It scores well in the demo — and then nobody can reproduce it, retrain it on fresh data, explain which data and code produced it, or tell when it has quietly gone stale in production. There is no version history, no approval before it ships, and no alert when the world drifts away from the data it learned on. This solution stands up **MLOps on Azure Machine Learning** so every model is built, registered, deployed, and monitored like software — reproducibly, with versioning and approvals, not a notebook copied by hand.

This is a complete, **deploy-it-today** package written for an administrator or ML engineer who has used the Azure portal and can run Python, but has never authored an Azure ML pipeline, promoted a model through a registry, or wired a train→register→deploy CI/CD flow.

What you get

• A plain-language **Solution Overview** of the MLOps pattern on Azure ML — workspace/compute/environment as code, reproducible training + evaluation pipelines with MLflow tracking, a versioned model registry with lineage and stage promotion, managed online/batch endpoints, and data-drift + model-performance monitoring.

• A **Requirements Guide** with verify / implement-if-missing for every solution-specific prerequisite (the category-wide ones live in the separate Global Requirements).

• An **Implementation Guide** of precise, ordered steps — provision the workspace and compute as **Bicep**, define the environment and training pipeline as code, run a reproducible train+evaluate job, register the candidate model, deploy behind an approval gate to a managed endpoint, then turn on drift and performance monitoring with alerts.

• Five automation artifacts across the right languages for the job: a read-only **PowerShell discovery** script, the `infra.bicep` workspace-and-compute template, a **Python** (`azure-ai-ml` SDK v2 + MLflow) reproducible **train→evaluate→register** pipeline, an `azure-pipelines.yml` CI/CD pipeline (train → register → **approval** → deploy), and a **PowerShell validation + posture** script.

• **Testing** and **Troubleshooting** guides for a confident, pilot-first rollout.

What this delivers

An Azure ML **workspace, compute, and environment defined as code**; a **reproducible training pipeline** whose every run is tracked in MLflow with the exact data, code, and metrics that produced the model; a **model registry** where each model is versioned with lineage and promoted Candidate → Staging → Production on a controlled gate; **managed online and batch endpoints** deployed only after a human approves the CI/CD gate; and **data-drift and model-performance monitors** that raise alerts when production data or accuracy moves away from the training baseline — proven in a non-production workspace before it reaches production.

The standard deployment model

Assess → **Configure** → **Pilot** → **Validate** → **Expand** — every training run is reproducible and tracked, every model promotion and every deploy is approval-gated, and monitoring is validated before you rely on it. The **decision to accept or reject a model** on its metrics stays with the data-science owner; the solution provides the evidence and the gate, not the verdict.

Outcome

An MLOps foundation on Azure Machine Learning where every model is **reproducible, versioned, approved, and monitored** in production — deployable by a single engineer, validated in a non-production workspace first, and repeatable for every model that follows.

1. Solution Overview — MLOps on Azure Machine Learning

What MLOps on Azure ML is

MLOps is the practice of building, shipping, and operating machine-learning models with the same discipline as software: everything is in source control, every result is reproducible, every change is reviewed and approved, and the running system is monitored. **Azure Machine Learning (Azure ML)** is Microsoft's managed platform for that lifecycle — a **workspace** that holds your compute, data, environments, jobs, models, and endpoints, all reachable from the `azure-ai-ml` Python SDK (v2), the Azure CLI `ml` extension, and the Azure ML Studio portal.

This solution assembles those pieces into a working MLOps loop. Instead of a model trained by hand in a notebook, you get: the **workspace and compute defined as code (Bicep)**; a **reproducible training + evaluation pipeline** whose runs are tracked by **MLflow**; a **model registry** that versions every model with its lineage; **CI/CD** that runs **train** → **register** → **deploy behind an approval**; **managed endpoints** for real-time and batch scoring; and **monitoring** that watches for data drift and model-performance decay and raises alerts.

How the technology works

- **Workspace** — the top-level Azure ML resource. It brings together a linked **Storage account** (data and artifacts), **Key Vault** (secrets), **Application Insights** (telemetry), and optionally a **Container Registry** (custom environments). You create it once, as code.
- **Compute as code** — training runs on a **compute cluster** (auto-scaling VMs that scale to zero when idle) and endpoints run on managed compute. Both are declared as code so they are reproducible and cost-controlled.
- **Environment as code** — an **environment** is a versioned, containerized definition of the exact Python/Conda dependencies a job runs in. Pinning it is what makes a run reproducible months later.
- **Jobs and pipelines** — a **command job** runs one script; a **pipeline job** chains steps (prep → train → evaluate → register) into a repeatable graph. Each run records its inputs, code, parameters, metrics, and outputs.
- **MLflow tracking** — Azure ML is MLflow-compatible. Your training code calls `mlflow.log_metric`/`log_param`/`log_model`, and the workspace stores that run's metrics, parameters, and the model artifact — so every model is traceable to the exact run, data, and code that

produced it (**lineage**).

• **Model registry** — registered models are **versioned** (`churn-classifier:1`, `:2`, …). Promotion between stages (**Candidate** → **Staging** → **Production**) is done with labels/tags and is the controlled gate before a model can be deployed. *MLflow's classic model "stages" are deprecated in newer MLflow; Azure ML uses model versions plus labels/registries — verify the current promotion mechanism in your workspace.*

• **Managed endpoints** — a **managed online endpoint** serves real-time scoring over HTTPS with blue/green deployments; a **managed batch endpoint** scores large datasets on a schedule or on demand. Neither is exposed publicly by default (private-by-default per the Global Requirements).

• **Model monitoring** — Azure ML **model monitoring** compares live production data against the training baseline and computes **data-drift** and **model-performance** signals on a schedule, firing alerts (via an Azure Monitor action group) when a signal crosses a threshold. *Model monitoring features move quickly and some are in preview — verify current availability in your tenant.*

What the MLOps foundation contains

1. **Workspace + compute + environment as code** — `infra.bicep` provisions the workspace and its linked resources and a training compute cluster; the environment is defined and versioned from the SDK.

2. **Reproducible train + evaluate pipeline** — a Python pipeline (SDK v2 + MLflow) that prepares data, trains, evaluates against a held-out set, and **conditionally registers** the model — with registration defaulting to a **plan/dry-run** until explicitly enabled.

3. **Model registry with versioning, lineage, and stage promotion** — every model version carries the run, data asset, and metrics that produced it; promotion Candidate → Staging → Production is a labelled, gated action.

4. **CI/CD train→register→deploy with approvals** — an `azure-pipelines.yml` (GitHub Actions equivalent noted) that trains, registers, and then deploys to the managed endpoint **only after a human approves** the environment gate.

5. **Managed endpoints + drift/performance monitoring** — an online (and optional batch) endpoint with blue/green rollout, plus data-drift and model-performance monitors wired to alerts.

What you can control

• **Model and data** — which training script, data asset, and evaluation metric/threshold define "good enough" (the threshold is a data-science decision the pipeline enforces, not decides).

• **Compute shape and cost** — VM SKU, min/max nodes, and scale-to-zero idle time on the training cluster.

• **Promotion policy** — which metric gate a model must clear to move Candidate → Staging → Production, and who approves it.

• **Deployment strategy** — online vs. batch, blue/green split, and the approval reviewers on the production gate.

• **Monitoring thresholds** — which features to watch for drift, the performance metric to track, the schedule, and the alert recipients.

• **Rollout scope** — a non-production workspace first, production only after validation and approval.

What is out of scope / limitations (be honest with buyers)

• This is the **MLOps platform and process**, not the data science. It does not choose your algorithm, engineer your features, or decide whether a model's accuracy is acceptable — **model-acceptance decisions belong to the data-science owner**; the solution supplies reproducibility, versioning, gates, and monitoring.

• **Model monitoring, registries, and some endpoint features may be in preview or vary by region and by Azure environment (Commercial vs. Government)** — *verify current availability in your tenant.* The SDK/CLI/MLflow and API versions used are marked to verify.

• It does not label or classify your training data, nor make the **data-classification, responsible-AI, or acceptable-use determinations** — those belong to the customer's compliance / legal / trust-and-safety functions (Global Requirements G6/G7).

• It secures the platform (private endpoints, managed identity, no keys) but does not replace application-level security review of the model's consumers.

• Retraining and production deployment are **approval-gated by design** — nothing retrains or ships to production without a human gate.

Authentication and connections used

• **Azure CLI (**`az` **+ the** `ml` **extension) /** `azure-ai-ml` **SDK v2** authenticated with `DefaultAzureCredential` (your signed-in identity locally; a managed identity or OIDC federation in CI/CD) — no stored keys.

• **Managed identity + RBAC** on the workspace and endpoints for keyless access to Storage, Key Vault, and the registry (Global Requirements G2) — with the **AzureML Data Scientist** / endpoint roles scoped least-privilege.

• **MLflow tracking URI** set to the Azure ML workspace, so tracking flows into the workspace with no separate MLflow server.

• A **CI/CD service connection** using **workload identity federation (OIDC)** — no secret — scoped to the workspace resource group (Global Requirements G4/G5).

The standard deployment model

> ***Assess*** *the current ML practice (where models live, how they're trained/deployed, what's tracked)* → ***configure*** *the workspace, compute, environment, training pipeline, registry, and endpoints as code* → ***pilot*** *by training and deploying in a* ***non-production workspace*** *with registration/deploy gated* → ***validate*** *with the testing gates (reproducible run, registered version with lineage, endpoint scoring, monitor firing)* → ***expand*** *to the production workspace behind pipeline approval, then turn on scheduled retraining and monitoring. The repeatable pattern:* ***Assess*** → ***Configure*** → ***Pilot*** → ***Validate*** → ***Expand****, reproducible-and-tracked, approval-gated, with a documented rollback (blue/green swap-back and prior model version).*

2. Requirements — MLOps on Azure Machine Learning

The category-wide prerequisites (an Azure subscription with AI-service access and rights, Entra identity + managed identity + RBAC, private-networking capability, IaC + Python tooling and source control, a CI/CD platform with OIDC, a data source with a known permission model, a content-safety / responsible-AI posture, and cost controls/quotas) are covered once in **Global Requirements — AI** and are **not repeated here**. Confirm those first. Below are the prerequisites **specific to standing up MLOps on Azure Machine Learning**. Each item: **Description / Why required / How to verify / How to implement if missing.**

R1. Azure ML CLI extension and `azure-ai-ml` SDK v2 at supported versions

Description: The Azure CLI `ml` **extension** and the Python `azure-ai-ml` SDK (v2) plus `mlflow` and `azureml-mlflow`, at versions that support pipelines, model monitoring, and managed endpoints. **Why required:** The pipeline, registry, endpoint, and monitoring steps are all driven by the v2 SDK/CLI; the v1 SDK and older CLI use a different, incompatible object model. **How to verify:** `az extension show -n ml` and `python -c "import azure.ai.ml, mlflow; print(azure.ai.ml.__version__, mlflow.__version__)"` return current builds; `az version` is current. *Verify current supported SDK/CLI/MLflow versions — the Azure AI SDKs move fast.* **How to implement if missing:** `az extension add -n ml` (or `az extension update -n ml`); `pip install -U azure-ai-ml mlflow azureml-mlflow`. Pin the versions in `requirements.txt` in the repo.

R2. Rights to create the Azure ML workspace and its linked resources

Description: The identity has **Contributor** (plus **User Access Administrator** for role assignments) on the target resource group so it can create the workspace and its linked Storage, Key Vault, Application Insights, and Container Registry. **Why required:** `infra.bicep` provisions the workspace and every linked resource and assigns managed-identity roles; without these rights the deploy fails. **How to verify:** `az role assignment list --assignee <you> --resource-group <rg>` shows Contributor; a `what-if` of `infra.bicep` completes without an authorization error. **How to implement if missing:** Have an Owner grant Contributor + User Access Administrator at the resource group. *Elevating scope is the customer's governance decision.*

R3. Non-production Azure ML workspace to pilot in

Description: A separate **non-production workspace** (or a clearly separate resource group) to run the first training, registration, endpoint, and monitoring cycle before touching production. **Why required:** The model pilots end-to-end in non-production first; deploying an unproven pipeline straight to a production workspace is exactly what MLOps exists to prevent. **How to verify:** A workspace named `ML Workspace – Platform – NonProd` (`mlw-platform-nonprod`) exists or can be created by `infra.bicep` with the non-prod parameters. **How to implement if missing:** Deploy `infra.bicep` with `env=nonprod` first (Implementation Step 2); only after validation deploy `env=prod`.

R4. A registered data asset (training data) with a known permission model

Description: The training dataset registered as an **Azure ML data asset** (a versioned `uri_folder`/`uri_file`/`mltable` pointing at the linked Storage), with the source's ACL/sensitivity understood. **Why required:** Reproducibility depends on the training run referencing a **versioned data asset**, not an ad-hoc file path; and the data's permissions/sensitivity must be respected (Global

Requirements G6). **How to verify:** `az ml data list -w <workspace> -g <rg>` shows the asset (e.g. `churn-training-data`); the discovery script reports data assets and their versions. **How to implement if missing:** Register the asset: `az ml data create --name churn-training-data --path <path> --type uri_folder -w <workspace> -g <rg>`. *Data-sensitivity classification is the compliance team's decision.*

R5. Managed identity and least-privilege data-plane roles for compute and endpoints

Description: The workspace, compute cluster, and endpoints use **managed identity** with least-privilege data-plane roles — e.g. **AzureML Data Scientist** for the training identity and **Storage Blob Data Contributor** on the linked storage — so jobs and endpoints authenticate without keys. **Why required:** Keys leak; keyless managed-identity auth is the private-by-default posture (Global Requirements G2). Missing roles cause jobs to fail reading data or writing artifacts. **How to verify:** The endpoint/compute has a system- or user-assigned identity (`az ml online-endpoint show ... --query identity`); the identity holds the data-plane roles on the storage/registry. **How to implement if missing:** Enable the identity in `infra.bicep`/at endpoint creation and assign the roles (the Bicep includes the role assignments; adjust the principal IDs).

R6. Model evaluation metric and acceptance threshold defined (by the data-science owner)

Description: A named **evaluation metric** (e.g. AUC, F1, RMSE) and a **numeric acceptance threshold** the candidate model must clear before it can be registered/promoted — decided by the data-science owner and encoded as a pipeline parameter. **Why required:** The pipeline's registration gate compares the evaluated metric to the threshold; without an agreed threshold the gate is meaningless. **The value and the accept/reject decision are the data-science owner's** — the pipeline only enforces the number. **How to verify:** The threshold is set in the pipeline parameters (e.g. `min_auc=0.80`) and recorded in the repo; a run below it does **not** register. **How to implement if missing:** Have the data-science owner set the metric and threshold; encode it as a parameter. *Model acceptance is deferred to the data-science owner — this guide provides the gate, not the verdict.*

R7. Pipeline approval gate and reviewer group for register/deploy

Description: A named group of approvers and a configured **environment approval** (Azure DevOps *Environment* checks, or a GitHub *Environment* with required reviewers) that the **register-to-production** and **deploy** stages must pass. **Why required:** No model is promoted to production or deployed to a production endpoint without human approval after the metrics are reviewed — this is the control that makes automated retraining safe. **How to verify:** The `mlops-prod` environment in the pipeline platform lists required reviewers; a test run pauses at the gate before deploy. **How to implement if missing:** Create the environment, add the reviewer group, and reference it from the register/deploy stages (shown in `azure-pipelines.yml`). *Who may approve a production model deploy is the customer's change-control decision.*

R8. Monitoring baseline dataset and an alert action group

Description: A **baseline (reference) dataset** — typically the training/validation data asset — for the drift monitor to compare production data against, plus an **Azure Monitor action group** to receive alerts (email/Teams/webhook). **Why required:** Data-drift monitoring is defined as "production vs. baseline"; without a registered baseline there is nothing to compare to, and without an action group the alerts have nowhere to go. **How to verify:** The baseline data asset exists (R4) and an action group (`Action Group – MLOps – Alerts`) is created (`az monitor action-group show`). **How to implement if missing:** Register the baseline asset and create the action group (`az monitor action-group create ...`); reference both when configuring the monitor (Implementation Step 8). *Who receives and triages alerts is the customer's operations decision.*

Global Requirements — AI (category-level)

These requirements apply to every solution in the AI category (building and operating custom AI on Azure — Azure OpenAI / AI Foundry, RAG, Azure ML / MLOps, AI landing zones, content safety). Published once at the category level so listings stay small. Confirm before deploying any AI solution. Each entry: Description / Why / Verify / Implement-if-missing.

> *Custom AI touches sensitive data and can emit harmful or ungrounded output. Deploy private-by-default (no public endpoints, keyless auth), ground and permission-trim on your own data, put safety guardrails in front of generation, and pilot in non-production before production — content-acceptability and AI-use-policy decisions belong to the customer's legal / trust-and-safety / compliance functions, not this guide.*

G1. Azure subscription with AI service access and rights

Description: An Azure subscription with access to the required AI services (Azure OpenAI / AI Foundry, Azure AI Search, Azure Machine Learning, Azure AI Content Safety) and Contributor/Owner at the deployment scope. Why: These solutions create and configure real, billable Azure AI resources. Verify: `az account show` succeeds; the AI resource providers are registered and the region is available. *Azure OpenAI/model access and regions vary — verify current availability in your subscription.* Implement: Obtain the subscription/role; request access to the gated AI services where required; pick a supported region.

G2. Entra identity, managed identity, and RBAC

Description: Entra ID identities plus managed identities for keyless service-to-service auth. Why: Keys leak; AI services should authenticate via managed identity + RBAC (least privilege). Verify: Managed identities can be assigned and the needed data-plane roles exist (e.g. Cognitive Services OpenAI User, Search Index Data Reader). Implement: Enable managed identity on the compute/app; assign least-privilege data-plane roles; store any unavoidable secret in Key Vault.

G3. Private networking capability

Description: A VNet with the ability to create private endpoints and private DNS for the AI services. Why: Public endpoints are a data-exfiltration surface; enterprise AI is private-by-default. Verify: The VNet/subnets and Private DNS zones exist or can be created; public access can be disabled. Implement: Provision the VNet, subnets, and Private DNS; plan private endpoints per service (see the AI landing-zone

solution).

G4. IaC + Python tooling and source control

Description: Terraform/Bicep, the Azure CLI, Python 3.x with the relevant Azure AI SDKs, and Git. Why: AI platforms are delivered as versioned code (infra + pipelines + app), so they are reproducible and reviewable. Verify: `az version`, `terraform version`/`az bicep version`, `python --version`, and `git --version` return supported builds. *Verify current SDK/provider versions — the AI SDKs move fast.* Implement: Install and pin the toolchain; store all config in Git.

G5. CI/CD platform

Description: A pipeline (GitHub Actions / Azure DevOps) with OIDC/workload-identity federation to Azure. Why: Infra, model, and app deployments must run through an auditable, approval-gated pipeline — not a laptop. Verify: The pipeline can authenticate to Azure via OIDC and run a plan/deploy step. Implement: Configure the service connection with workload-identity federation; scope its rights.

G6. Data source and classification (for data-grounded solutions)

Description: The private data to ground on (documents, databases) with a known permission/ACL model, and a sensitivity-classification position. Why: RAG must respect source permissions (permission-trimming) and must not expose sensitive data through AI. Verify: The data source, its access model, and applicable sensitivity labels are known. Run the solution's discovery script. Implement: Inventory the data and its ACLs; align sensitivity labeling (see the Purview solutions). *Data-classification determinations are the compliance team's decision.*

G7. Content-safety and responsible-AI posture

Description: An organizational position on acceptable AI content, safety thresholds, and responsible-AI review. Why: Generative AI can emit harmful/ungrounded output and be jailbroken; the organization must define guardrails and review. Verify: A responsible-AI / acceptable-use policy exists; content-safety is enabled where generation occurs. Implement: Adopt an RAI/acceptable-use policy and enable content safety (see the content-safety solution). *Acceptable-content and RAI determinations are the customer's legal / trust-and-safety decision.*

G8. Cost controls and quotas

Description: Budgets, alerts, and per-service quotas (e.g. Azure OpenAI TPM) appropriate to the workload. Why: AI services bill on tokens/compute; unmanaged usage causes bill shock. Verify: Budgets/alerts exist and quotas are set for the deployed services. Implement: Set budgets, cost alerts, and quotas/rate limits before broad rollout. *Spend limits are the customer's business decision.*

The deployment model (all solutions): Assess -> Configure -> Pilot -> Validate -> Expand.

3. Implementation — MLOps on Azure Machine Learning

Follow these steps in order. Everything is **pilot-first**: you build and prove the whole loop in a **non-production workspace** with registration and deploy **gated**, validate, then expand to production behind pipeline approval. Every object you create is named with the convention `<Purpose> – <Scope> – <Stage>` and that exact name is reused in later steps, testing, and the scripts. The worked example is a benign **customer-churn classifier**; substitute your own model. Placeholder tenant is `contoso.com` / `contoso.onmicrosoft.com`.

> *Run every training run so it is **reproducible and tracked** (versioned data asset + pinned environment + MLflow). Never register or deploy to production without the metric gate **and** the pipeline approval. All examples use the Azure CLI `ml` extension and the `azure-ai-ml` SDK v2; the pipeline runs the same commands.*

Naming you will create (reuse these exact names)

Object	Name
Non-prod workspace	`ML Workspace – Platform – NonProd` (`mlw-platform-nonprod`)
Prod workspace	`ML Workspace – Platform – Prod` (`mlw-platform-prod`)
Training compute cluster	`Training Cluster – Platform – NonProd` (`cpu-train-nonprod`)
Training environment (as code)	`Training Environment – Sklearn – v1` (`train-env-sklearn`)
Registered training data asset	`churn-training-data` (versioned)
Train + evaluate pipeline job	`Train-Eval Pipeline – Churn – NonProd`
Registered model	`churn-classifier` (versions `:1`, `:2`, …)
Model stage labels	`Candidate` → `Staging` → `Production`
Managed online endpoint	`Online Endpoint – Churn – Prod` (`churn-online-prod`)
Online deployment (blue/green)	`Deployment – Churn – Blue` / `Deployment – Churn – Green`
Managed batch endpoint	`Batch Endpoint – Churn – Prod` (`churn-batch-prod`)
Data-drift monitor	`Monitor – Churn – Drift`
Model-performance monitor	`Monitor – Churn – Perf`
Alert action group	`Action Group – MLOps – Alerts`
CI/CD pipeline	`MLOps Pipeline – Churn – CICD`
Pipeline prod environment (gate)	`mlops-prod`

Step 1 — Assess: inventory the current ML practice (read-only)

1. Sign in: `az login` (add `--tenant contoso.onmicrosoft.com` if you have several) and `az account set --subscription "<non-prod sub id>"`.

2. Run the discovery script from the `scripts/` folder:

`powershell -ExecutionPolicy Bypass -File .\AIMLOPS-Discovery.ps1 -ResourceGroup <rg>` It reports any existing Azure ML workspaces, compute, registered models and versions, data assets, endpoints, and monitors. Save the output — it is your baseline and confirms **R3/R4**.

1. Confirm your identity has the rights from **R2** on the target resource group and that the `ml` extension + SDK are at supported versions (**R1**).

Step 2 — Configure: provision the workspace and compute as code (non-prod first)

1. Clone the empty repo (Global Requirements G4) and add the provided files: `infra.bicep`, `train_register_pipeline.py`, `azure-pipelines.yml` at the root, and `AIMLOPS-Discovery.ps1` / `AIMLOPS-Validate.ps1` under `scripts/`. Add a `requirements.txt` pinning `azure-ai-ml`, `mlflow`, `azureml-mlflow`, `scikit-learn` (**R1**).

2. **Preview with what-if** (nothing is applied):

```
az deployment group what-if -g <rg> --template-file infra.bicep --parameters env=nonprod workspaceName=mlw-platform-nonprod
```

1. Read the preview — it should create the workspace `ML Workspace – Platform – NonProd` plus its linked Storage, Key Vault, Application Insights, and Container Registry, and the training cluster `Training Cluster – Platform – NonProd` (scale-to-zero).

2. Deploy: `az deployment group create -g <rg> --template-file infra.bicep --parameters env=nonprod workspaceName=mlw-platform-nonprod`.

3. Set the workspace defaults so later commands are shorter: `az configure --defaults workspace=mlw-platform-nonprod group=<rg>`.

4. Commit to a **feature branch** and open a pull request — infrastructure and pipeline changes are reviewed, never pushed straight to `main`.

Step 3 — Configure: register the data asset and define the environment as code

1. Register the training data as a **versioned data asset** (**R4**):

`az ml data create --name churn-training-data --path ./data/churn --type uri_folder` Reuse the name `churn-training-data` everywhere; the run pins a specific version for reproducibility.

1. Create the **environment as code** so the run is reproducible: define `Training Environment – Sklearn – v1` from a conda file:

`az ml environment create --name train-env-sklearn --version 1 --conda-file ./env/conda.yml --image mcr.microsoft.com/azureml/openmpi4.1.0-ubuntu20.04` *Verify the current base image tag in your region.* Pin the environment version in the pipeline — never let it float.

Step 4 — Configure: run the reproducible train + evaluate pipeline (registration in DRY-RUN)

1. Run the training pipeline in its default **plan / dry-run** mode — it trains and evaluates but does **not** register:

`python train_register_pipeline.py --workspace mlw-platform-nonprod --resource-group <rg> --data-asset churn-training-data --min-auc 0.80` This submits the pipeline job `Train-Eval Pipeline – Churn – NonProd` (prep → train → evaluate), tracked in MLflow.

1. Open **Azure ML Studio** → **Jobs** and confirm the run recorded its **parameters, metrics (AUC/F1), and the model artifact**, and that its **lineage** links back to `churn-training-data` and the pinned environment.

2. Note the evaluated metric. The registration gate compares it to `--min-auc` (the threshold from **R6**). **The accept/reject decision is the data-science owner's** — the pipeline only enforces the number.

Step 5 — Configure: register the candidate model (gated on the metric)

1. Once the data-science owner accepts the metric, re-run with registration enabled:

`python train_register_pipeline.py --workspace mlw-platform-nonprod --resource-group <rg> --data-asset churn-training-data --min-auc 0.80 --register` If the evaluated metric clears `--min-auc`, the pipeline registers `churn-classifier` as a new version and labels it `Candidate`; if it does not clear the threshold, **registration is skipped** and the run is marked failed-gate.

1. Confirm the version and its lineage: `az ml model list --name churn-classifier -o table` and open the model in Studio to see the **run, data asset, and metrics** attached (this is the registry lineage).

2. Promote Candidate → **Staging** by moving the label (a controlled action): `az ml model update --name churn-classifier --version <n> --set tags.stage=Staging`. *MLflow classic stages are deprecated; Azure ML uses versions + labels/tags — verify the current promotion mechanism in your workspace.*

Step 6 — Pilot: deploy to a managed online endpoint (blue/green)

1. Create the endpoint (non-prod first): `az ml online-endpoint create --name churn-online-prod --auth-mode aad_token` — display name `Online Endpoint – Churn – Prod`, keyless (managed identity, **R5**), no public network access.

2. Deploy the Staging model as the **blue** deployment with a small instance count:

`az ml online-deployment create --name blue --endpoint churn-online-prod --model churn-classifier:<n> --instance-count 1` — this is `Deployment – Churn – Blue`.

1. Send **0% live traffic** first, smoke-test with a sample payload (`az ml online-endpoint invoke --name churn-online-prod --request-file ./sample-request.json`), then shift traffic: `az ml online-endpoint update --name churn-online-prod --traffic "blue=100"`.

2. For a later model, deploy `Deployment – Churn – Green` alongside blue, validate at 0%, then swap traffic — your rollback is shifting traffic back to blue.

3. *(Optional)* Create the **batch endpoint** for scheduled/large scoring: `az ml batch-endpoint create --name churn-batch-prod` (`Batch Endpoint – Churn – Prod`) and a deployment referencing the same model version.

Step 7 — Wire the CI/CD pipeline (train → register → approval → deploy)

1. In Azure DevOps (or GitHub), create the pipeline `MLOps Pipeline – Churn – CICD` from `azure-pipelines.yml`.

2. Confirm the **service connection** uses workload identity federation (Global Requirements G5) and the `mlops-prod` **environment** has the approval gate and reviewer group from **R7**.

3. The pipeline stages are: **Validate** (lint, `requirements.txt`, unit tests) → **Train+Evaluate** (submits the pipeline job, publishes metrics) → **Register** (labels the version `Candidate`/`Staging` if the metric gate passes) → **Approval** (manual gate on `mlops-prod`) → **Deploy** (blue/green to the managed endpoint). Merging to `main` runs Validate → Train → Register; **Deploy only runs after approval**.

Step 8 — Configure: turn on drift and performance monitoring with alerts

1. Create the alert `Action Group – MLOps – Alerts`: `az monitor action-group create --name mlops-alerts --resource-group <rg> --short-name mlops --action email ops <ops@contoso.com>`.

2. Configure the **data-drift monitor** `Monitor – Churn – Drift` comparing the online endpoint's collected production data against the **baseline** (`churn-training-data`, **R8**), on a schedule, alerting the action group when drift crosses the threshold. *Enable production-data collection on the deployment first; model monitoring is configured via `az ml schedule` / the monitoring YAML — verify current availability, some monitoring features are in preview.*

3. Configure the **model-performance monitor** `Monitor – Churn – Perf` to track the production metric (once ground-truth labels are available) against the training baseline, alerting on decay.

4. Confirm both monitors appear under **Studio** → **Monitoring** and that a forced run reaches the action group (Testing Test 7).

Step 9 — Validate, then Expand: production workspace behind approval

1. Run the validation script against non-prod: `powershell -ExecutionPolicy Bypass -File .\AIMLOPS-Validate.ps1 -Workspace mlw-platform-nonprod -ResourceGroup <rg> -ModelName churn-classifier` and clear every gate in the **Testing** guide.

2. Deploy the **production workspace**: `az deployment group create -g <prod-rg> --template-file infra.bicep --parameters env=prod workspaceName=mlw-platform-prod` (what-if first).

3. Let the pipeline run **Validate** → **Train** → **Register** against production, review the metrics and what the Register stage will label, **approve** the `mlops-prod` gate, and let **Deploy** roll out blue/green to `Online Endpoint – Churn – Prod` in the production workspace.

4. Enable the **scheduled retrain** trigger (pipeline schedule or new-data trigger) — every retrain still runs the same metric gate and approval before it can replace the Production model.

5. **Rollback** at any point: shift endpoint traffic back to the previous **blue** deployment and/or re-deploy the prior registered model version (`churn-classifier:<n-1>`). The registry keeps every version, so rollback is a redeploy, not a rebuild.

Onboard the next model by pointing the same pipeline at a new training script, data asset, and threshold — it inherits the workspace, registry, endpoint pattern, gates, and monitoring automatically. That is the repeatable payoff of the MLOps foundation.

5. Testing — MLOps on Azure Machine Learning

Validate the MLOps loop against the **non-production workspace** before you expand to production. Each test has an explicit **pass/fail gate**. Reference the scripts by name; they are read-only / plan-only except where noted. Run `AIMLOPS-Validate.ps1` after any fix to re-check the gates.

Test 1 — Workspace and compute exist as code

• **How:** `powershell -ExecutionPolicy Bypass -File .\AIMLOPS-Validate.ps1 -Workspace mlw-platform-nonprod -ResourceGroup <rg> -ModelName churn-classifier`, or in Studio open **Compute**.

• **Pass:** `ML Workspace – Platform – NonProd` exists with its linked Storage/Key Vault/App Insights, and `Training Cluster – Platform – NonProd` is present and scales to zero when idle.

• **Fail:** Workspace or cluster missing → re-run Step 2's `infra.bicep` what-if and check your **R2** rights.

Test 2 — A training run is reproducible and tracked

• **How:** Submit `Train-Eval Pipeline – Churn – NonProd` (Step 4) and open **Studio** → **Jobs**.

• **Pass:** The run recorded its **parameters, metrics (AUC/F1), and model artifact**, and its **lineage** links to the `churn-training-data` asset version and the pinned `train-env-sklearn:1` environment. Re-submitting with the same inputs reproduces the metrics within tolerance.

• **Fail:** No metrics/artifact logged → confirm the script calls `mlflow.log_*` and the tracking URI is the workspace. No lineage → confirm the run consumed the **registered data asset**, not a raw path.

Test 3 — Registration is gated on the metric threshold

• **How:** Run the pipeline once with a threshold **above** the model's metric (e.g. `--min-auc 0.99 --register`), then once at the agreed threshold (`--min-auc 0.80 --register`).

• **Pass:** The above-threshold run **does not register** (fails the gate); the at-threshold run registers a new `churn-classifier` version labelled `Candidate`.

• **Fail:** A below-threshold run still registers → the gate is not enforced; check the `--min-auc` comparison in `train_register_pipeline.py`. (The threshold value and accept/reject remain the **data-science owner's** decision — the test only confirms the gate fires.)

Test 4 — Model registry versioning and lineage

• **How:** `az ml model list --name churn-classifier -o table`; open a version in Studio.

• **Pass:** Each registered model is a distinct **version** with the run, data asset, and metrics attached, and a `stage` label (`Candidate`/`Staging`/`Production`) you can move on a gate.

• **Fail:** Versions overwrite instead of increment → you are re-registering with a fixed version; let Azure ML auto-version. Lineage empty → see Test 2.

Test 5 — Managed online endpoint scores

• **How:** `az ml online-endpoint invoke --name churn-online-prod --request-file ./sample-request.json`.

• **Pass:** `Online Endpoint – Churn – Prod` returns a scored response from the **blue** deployment; the endpoint is keyless (`aad_token`) and has no public network access.

• **Fail:** `403`/auth error → the caller's identity lacks the endpoint invoke role (**R5**). `5xx` → check the deployment logs (`az ml online-deployment get-logs`) and the scoring script/environment.

Test 6 — Blue/green rollback works

• **How:** Deploy `Deployment – Churn – Green` at 0% traffic, validate it, shift `green=100`, then shift back `blue=100`.

• **Pass:** Traffic moves without downtime and shifting back to **blue** restores the previous model — rollback is a traffic swap, not a rebuild.

• **Fail:** Swap causes errors → green's model/env is broken; keep traffic on blue and fix green before shifting.

Test 7 — Drift/performance monitors fire and alert

• **How:** With production-data collection enabled, force a monitor run (or feed skewed sample data) and check **Studio** → **Monitoring** and the action group.

• **Pass:** `Monitor – Churn – Drift` and `Monitor – Churn – Perf` produce a result, and a threshold breach delivers an alert to `Action Group – MLOps – Alerts`.

• **Fail:** Monitor `NotStarted` → data collection not enabled or no baseline (**R8**). No alert → the action group is not attached or the threshold was not crossed. *Some monitoring features are in preview — verify availability.*

Test 8 — Pipeline enforces the metric gate and approval

• **How:** Push a trivial change on a branch; open a PR; watch `MLOps Pipeline – Churn – CICD`.

• **Pass: Validate** → **Train+Evaluate** → **Register** run automatically and publish metrics; the **Deploy** stage **pauses at the** `mlops-prod` **approval gate** (**R7**) and only proceeds after an approver signs off.

• **Fail:** Deploy runs without pausing → the environment approval check is not attached to the Deploy stage; fix in the pipeline environment configuration.

Exit criteria

All eight tests pass against the non-production workspace, a model is registered with lineage and clears the agreed metric gate, the endpoint scores and rolls back cleanly, the monitors fire and alert, and the pipeline pauses for approval — **only then** deploy the production workspace and enable scheduled retraining. The decision that the model is **good enough to ship** stays with the data-science owner.

6. Troubleshooting — MLOps on Azure Machine Learning

Common failures during an Azure ML MLOps rollout, as **symptom** → **cause** → **fix**. Re-run `AIMLOPS-Validate.ps1` after any fix to re-check the gates.

Tooling / SDK

Symptom: `az ml ...` fails with `az: 'ml' is not in the 'az' command group`, or Python raises `ModuleNotFoundError: azure.ai.ml`. **Cause:** The CLI `ml` extension or the `azure-ai-ml` SDK is not installed / too old (**R1**). **Fix:** `az extension add -n ml` (or `az extension update -n ml`); `pip install -U azure-ai-ml mlflow azureml-mlflow`. Pin them in `requirements.txt`. *Verify current supported versions.*

Symptom: Code written against the old SDK (v1 `Workspace.from_config()`, `Experiment`, `Run`) errors against this solution. **Cause:** Mixing SDK v1 and v2 object models. **Fix:** Use only the v2 `MLClient` surface shown in `train_register_pipeline.py`; do not import `azureml.core`.

Workspace / infrastructure deploy

Symptom: `infra.bicep` fails with `AuthorizationFailed` on the workspace or role assignment. **Cause:** The identity lacks Contributor (+ User Access Administrator for the role assignment) on the resource group (**R2**). **Fix:** Have an Owner grant the roles; re-run the `what-if` then deploy.

Symptom: Workspace create fails referencing a missing Storage/Key Vault/App Insights/ACR, or a name conflict. **Cause:** A linked resource name collided (they are globally scoped for some types) or a provider is unregistered. **Fix:** The Bicep derives unique names via `uniqueString`; if a name still collides, adjust the prefix. Register providers: `az provider register --namespace Microsoft.MachineLearningServices` (and `Microsoft.Storage`, `Microsoft.KeyVault`, `Microsoft.ContainerRegistry`).

Training run / reproducibility

Symptom: A training job fails with "cannot access data" or a storage `403`. **Cause:** The compute/workspace managed identity lacks the data-plane role on the linked storage (**R5**). **Fix:** Confirm the **Storage Blob Data Contributor** assignment from `infra.bicep` targeted the right identity; re-assign if the principal ID changed.

Symptom: Re-running the same pipeline produces different metrics, or a run has **no lineage**. **Cause:** The run read a raw file path instead of the **registered data asset**, or the **environment version floated** instead of being pinned. **Fix:** Point the input at `azureml:churn-training-data@latest` (or a pinned version) and reference the pinned `train-env-sklearn:1` environment — never an unpinned image (Best

Practices).

Symptom: The job stays queued and never starts. **Cause:** The training cluster is scaled to zero and is spinning up a node, or `maxNodeCount` / quota is exhausted. **Fix:** Wait for scale-up (expected on first run), or raise the cluster max nodes / request quota (Global Requirements G8). This is normal cost-saving behaviour, not a failure.

Registration / metric gate

Symptom: A model with a poor metric still got registered. **Cause:** The pipeline was run with a threshold set too low, or the gate comparison was bypassed. **Fix:** Confirm `--min-auc` is the value the **data-science owner** set (**R6**) and that the run used `--register` only after review. The script skips registration when the metric is below threshold or `NaN` — check the evaluate step actually logged the metric via MLflow.

Symptom: `--register` did nothing and the run "passed". **Cause:** This is the **dry-run** path — without `--register` the script trains, evaluates, and prints the gate decision but registers nothing (by design). **Fix:** Re-run with `--register` once the metric is accepted (Implementation Step 5).

Symptom: Registered versions overwrite each other instead of incrementing. **Cause:** A fixed version was passed to registration. **Fix:** Let Azure ML auto-version — do not hard-code a model version on register.

Model promotion

Symptom: Setting an MLflow model "stage" (Staging/Production) errors or is ignored. **Cause:** MLflow's classic model **stages are deprecated**; Azure ML promotes via **model versions + labels/tags**, not the old stage API. **Fix:** Promote by moving the `stage` tag/label (`az ml model update ... --set tags.stage=Staging`). *Verify the current promotion mechanism in your workspace.*

Endpoints

Symptom: `az ml online-endpoint invoke` returns `401/403`. **Cause:** The endpoint is keyless (`aad_token`) and the caller's identity lacks the invoke role (**R5**). **Fix:** Grant the caller the endpoint's data-plane invoke role; use `az ml online-endpoint invoke` with an authenticated context.

Symptom: A new deployment returns `5xx` after traffic shift. **Cause:** The scoring script or environment in the new (green) deployment is broken. **Fix:** Keep traffic on **blue** (rollback is a traffic swap), read `az ml online-deployment get-logs`, fix green, validate at 0%, then shift again (Testing Test 6).

Symptom: Deployment fails on quota / instance count. **Cause:** Endpoint compute quota exhausted for the VM SKU. **Fix:** Lower `--instance-count`, choose an available SKU, or request quota. Remember online compute bills while it runs.

Monitoring

Symptom: A drift monitor stays `NotStarted` or produces no result. **Cause:** Production-data collection is not enabled on the deployment, or there is no **baseline** dataset to compare against (**R8**). **Fix:** Enable data collection on the online deployment and set the baseline to `churn-training-data`; force a monitor run. *Some model-monitoring features are in preview — verify availability.*

Symptom: Monitors run but no alert arrives. **Cause:** The **action group** is not attached, or the threshold was never crossed. **Fix:** Attach `Action Group – MLOps – Alerts` to the monitor and confirm the recipient; test with skewed sample data (Testing Test 7).

CI/CD pipeline

Symptom: The Deploy stage runs without pausing for approval. **Cause:** The Environment approval check (required reviewers) is not attached to the Deploy stage (**R7**). **Fix:** Add required reviewers to the `mlops-prod` Environment and confirm the deploy job references `environment: 'mlops-prod'` — re-run (Testing Test 8).

Symptom: Pipeline auth fails with a token/credential error. **Cause:** The service connection is not configured for workload identity federation, or the federated subject/scope is wrong (**G5**). **Fix:** Reconfigure the service connection for OIDC/workload identity federation and scope it to the workspace resource group; never fall back to a stored secret.

7. Best Practices — MLOps on Azure Machine Learning

Keep it clean

• **Reproducible or it didn't happen.** Every training run pins a **versioned data asset** and a **pinned environment**, and logs to MLflow. If a run reads a raw file path or a floating environment, you cannot reproduce or defend the model it produced.

• **The registry is the source of truth, not a folder.** Register every model; never hand-copy a `.pkl` to a server. Deploy **from a registered version**, and keep old versions — they are your rollback.

• **Gate on the metric, decide with a human.** The pipeline enforces the numeric threshold (**R6**), but **whether the model is acceptable is the data-science owner's call.** Keep the accept/reject decision human; keep the enforcement automated.

• **Approval before production, always.** No model reaches a production endpoint without the `mlops-prod` approval after the metrics are reviewed. Automated retraining is safe *because* it still stops at the gate.

• **Pilot in a non-production workspace.** Prove the whole loop — train, register, deploy, monitor — in non-prod before production. It is far cheaper to find a broken scoring script there.

• **Blue/green every deploy.** Deploy new models at **0% traffic**, smoke-test, then shift. Rollback is a traffic swap back to the previous deployment, not a rebuild.

• **Pin the toolchain.** Pin `azure-ai-ml`, `mlflow`, `azureml-mlflow`, and the CLI `ml` extension in `requirements.txt`. ML SDKs move fast; a silent upgrade breaks pipelines. *Verify current supported versions periodically.*

• **Keyless by default.** Managed identity + least-privilege data-plane roles for compute and endpoints (**R5**); no keys, no public network access. Secrets that are unavoidable live in the workspace Key Vault.

• **Scale to zero.** Set the training cluster's idle-scale-down so it costs nothing when not training (Global Requirements G8). Watch endpoint instance counts — real-time compute bills while it runs.

• **Monitor from day one.** Turn on data-drift and performance monitoring with alerts as part of go-live, not later. A model with no monitoring is a model you will trust long after it has quietly gone stale.

• **Everything through the pipeline.** After bootstrap, don't train/register/deploy to production from a laptop. The pipeline's Validate → Train → Register → approval → Deploy path is the only production route, so every change is reviewed and recorded.

Decisions that belong to the customer

• **Model acceptance** — whether a model's metrics are good enough to register, promote, and ship — is the **data-science owner's** decision. The solution provides the reproducibility, the gate, and the evidence.

• **Production change approval**, who may approve a deploy, and the retrain cadence are the customer's **change-control** decisions.

• **Data classification, responsible-AI, and acceptable-use** determinations belong to the customer's compliance / legal / trust-and-safety functions (Global Requirements G6/G7) — the platform enforces controls once those choices are made.

• **Alert triage ownership** — who receives and acts on drift/performance alerts — is the customer's operations decision.

Where to go next

Once the MLOps loop is live, adjacent catalog areas extend it:

• **AI** — the *AI Landing Zone (Governance, Private Networking & Cost)* solution wraps this workspace in a governed, private-by-default landing zone; *Azure OpenAI / RAG* solutions if you add generative models alongside the classical ones.

• **DevOps** — the *Azure Landing Zone (Bicep / Terraform)* solutions to put the workspace's subscription under management-group policy and CI/CD discipline.

• **Security / Identity** — Conditional Access and PIM solutions to harden who can approve deploys and access the workspace.

• **Data** — the *Microsoft Fabric / Data platform* solutions to feed governed, versioned data into the training pipeline upstream.

8. Deliverables Summary — MLOps on Azure Machine Learning

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box

Document	Purpose
1. Solution Overview	What MLOps on Azure ML is — workspace/compute/environment as code, reproducible pipelines + MLflow, model registry with versioning/lineage/stage promotion, managed endpoints, drift/performance monitoring; how the technology works; scope and limitations
2. Requirements	Solution-specific prerequisites with Verify / Implement-if-missing
Global Requirements (AI)	Category-wide prerequisites, published once
3. Implementation	Exact, ordered steps — provision workspace/compute as Bicep, register data + environment, run the reproducible train+evaluate pipeline, register on the metric gate, deploy blue/green to a managed endpoint, wire CI/CD, turn on monitoring, then prod behind approval
5. Testing	Validation procedures with pass/fail gates
6. Troubleshooting	Field issues → cause → fix
7. Best Practices	Field-proven recommendations and where to go next

Automation artifacts

File	What it does
`AIMLOPS-Discovery.ps1`	Read-only discovery — existing Azure ML workspaces, compute, registered models and versions, data assets, endpoints, and monitors; your pre-deployment baseline (**R3/R4**)
`infra.bicep`	The core infrastructure-as-code template — the Azure ML workspace and its linked Storage / Key Vault / Application Insights / Container Registry, the scale-to-zero training compute cluster, and managed-identity role assignments; deployed what-if-first for non-prod then prod
`train_register_pipeline.py`	The reproducible **train** → **evaluate** → **register** pipeline in Python (`azure-ai-ml` SDK v2 + MLflow) — prep/train/evaluate steps, MLflow tracking and lineage, and a **metric-gated register that defaults to a dry-run** (registers only with `--register` and only if the evaluated metric clears the threshold)

File	What it does
`azure-pipelines.yml`	CI/CD pipeline (Azure DevOps; GitHub Actions notes inline) — Validate → Train+Evaluate → Register → **approval gate** → Deploy (blue/green), metric-gated and approval-gated
`AIMLOPS-Validate.ps1`	Read-only / plan-only validation + posture — confirms the workspace and compute, a registered model version with lineage, endpoint health and traffic split, and monitor status against the Testing guide's pass/fail gates (also surfaces the registry and monitoring report)

> *Note on the five artifacts: this solution's automation spans the right languages for each job — **Bicep** for infrastructure, **Python (SDK v2 + MLflow)** for the training/registration pipeline, **YAML** for CI/CD, and **PowerShell** for read-only discovery and validation. The validation script also carries the registry/endpoint/monitoring reporting, so a separate report script is not needed.*

What the buyer receives

- This implementation guide as a **Word** document and **PDF**.
- All five automation artifacts (`infra.bicep`, `train_register_pipeline.py`, `azure-pipelines.yml`, and two PowerShell scripts) in a single ZIP.
- A repeatable process following the model: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** — reproducible-and-tracked, metric-gated, approval-gated, with blue/green and prior-version rollback.

Delivered as a Complete Solution Package (PDF + Word + scripts).

Outcome

An MLOps foundation on Azure Machine Learning where every model is **reproducible, versioned, approved, and monitored** in production — proven in a non-production workspace before it reaches production, with model-acceptance decisions kept where they belong, with the data-science owner.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

AIMLOPS-Discovery.ps1

Purpose: Discovery for the "MLOps on Azure Machine Learning" solution. Reports existing Azure ML workspaces, compute (clusters/instances), registered models and their versions, registered data assets, online/batch endpoints, and any configured model monitors. Makes NO changes — this is your baseline snapshot and confirms requirements R3 (a non-prod workspace) and R4 (a registered training data asset). HOW TO RUN powershell -ExecutionPolicy Bypass -File .\AIMLOPS-Discovery.ps1 -ResourceGroup <rg> powershell -ExecutionPolicy Bypass -File .\AIMLOPS-Discovery.ps1 -ResourceGroup <rg> -Workspace mlw-platform-nonprod Requires the Azure CLI with the 'ml' extension (az extension add -n ml). Run 'az login' first. Verify current az CLI and ml-extension versions — the v2 CLI surface changes between releases.* EXPECTED RESULTS A per-section snapshot: workspaces, compute, models + versions, data assets, endpoints, and monitors — a single readiness picture before you deploy infra.bicep and run the pipeline. .NOTES White-label. Read-only — reports on configuration/identifiers only; makes no changes. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$ResourceGroup, [string]$Workspace, [string]$SubscriptionId)

How to run: powershell -ExecutionPolicy Bypass -File .\AIMLOPS-Discovery.ps1 -ResourceGroup <rg> powershell -ExecutionPolicy Bypass -File .\AIMLOPS-Discovery.ps1 -ResourceGroup <rg> -Workspace mlw-platform-nonprod Requires the Azure CLI with the 'ml' extension (az extension add -n ml). Run 'az login' first. Verify current az CLI and ml-extension versions — the v2 CLI surface changes between releases.* EXPECTED RESULTS A per-section snapshot: workspaces, compute, models + versions, data assets, endpoints, and monitors — a single readiness picture before you deploy infra.bicep and run the pipeline. .NOTES White-label. Read-only — reports on configuration/identifiers only; makes no changes. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$ResourceGroup, [string]$Workspace, [string]$SubscriptionId)

AIMLOPS-Validate.ps1

Purpose: Confirms the MLOps foundation matches the Testing guide's pass/fail gates and prints a posture report: the workspace and training compute exist, a model version is registered with lineage, the online endpoint is keyless + has a healthy blue deployment and a valid traffic split, and drift/performance monitors are present. Also surfaces the registry (versions + stage labels) and monitor status as a report. Makes NO changes — read-only / plan-only. This is both the validation and the reporting artifact for the solution. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\AIMLOPS-Validate.ps1 ` -Workspace mlw-platform-nonprod -ResourceGroup <rg> -ModelName churn-classifier Optional: -OnlineEndpoint churn-online-prod -OutFile posture.csv Requires the Azure CLI with the 'ml' extension. Run 'az login' first. Verify current az CLI / ml-extension versions.* EXPECTED RESULTS A PASS/FAIL line per gate, a registry/endpoint/monitor posture report, and an optional CSV. Exit code 0 when all gates pass, 1 otherwise. No changes are made. .NOTES White-label. Read-only / plan-only. Does not deploy. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$Workspace, [Parameter(Mandatory = $true)][string]$ResourceGroup, [string]$ModelName = 'churn-classifier', [string]$OnlineEndpoint = 'churn-online-prod', [string]$TrainingCluster, [string]$OutFile)

How to run: powershell -ExecutionPolicy Bypass -File .\AIMLOPS-Validate.ps1 ` -Workspace mlw-platform-nonprod -ResourceGroup <rg> -ModelName churn-classifier Optional: -OnlineEndpoint

churn-online-prod -OutFile posture.csv Requires the Azure CLI with the 'ml' extension. Run 'az login' first. Verify current az CLI / ml-extension versions.* EXPECTED RESULTS A PASS/FAIL line per gate, a registry/endpoint/monitor posture report, and an optional CSV. Exit code 0 when all gates pass, 1 otherwise. No changes are made. .NOTES White-label. Read-only / plan-only. Does not deploy. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$Workspace, [Parameter(Mandatory = $true)][string]$ResourceGroup, [string]$ModelName = 'churn-classifier', [string]$OnlineEndpoint = 'churn-online-prod', [string]$TrainingCluster, [string]$OutFile)

azure-pipelines.yml

Purpose: Metric-gated, approval-gated train -> register -> deploy. Stages: Validate -> lint + install pinned requirements + unit tests TrainEval -> submit the reproducible pipeline job (prep/train/evaluate) Register -> metric-gated register (labels the version Candidate/Staging) Deploy -> blue/green deploy to the managed online endpoint, BEHIND a manual approval on the 'mlops-prod' Environment (reviewers = R7) HOW TO RUN Create the pipeline "MLOps Pipeline – Churn – CICD" from this file. Set the service connection (workload identity federation — no secret, G5) below, and add required reviewers to the 'mlops-prod' Environment so Deploy pauses for approval. Retraining is safe because every retrain still hits this gate. EXPECTED RESULTS PRs run Validate + TrainEval + Register automatically and publish metrics. Deploy only runs after a human approves the Environment check. Nothing ships to a production endpoint unreviewed, and a below-threshold model never registers. White-label. Metric-gated + approval-gated. GitHub Actions equivalent noted at the bottom. *Verify current task versions, az 'ml' extension, and SDK versions.*
==========

How to run: Create the pipeline "MLOps Pipeline – Churn – CICD" from this file. Set the service connection (workload identity federation — no secret, G5) below, and add required reviewers to the 'mlops-prod' Environment so Deploy pauses for approval. Retraining is safe because every retrain still hits this gate. EXPECTED RESULTS PRs run Validate + TrainEval + Register automatically and publish metrics. Deploy only runs after a human approves the Environment check. Nothing ships to a production endpoint unreviewed, and a below-threshold model never registers. White-label. Metric-gated + approval-gated. GitHub Actions equivalent noted at the bottom. *Verify current task versions, az 'ml' extension, and SDK versions.*
==========

infra.bicep

Purpose: // Declares the MLOps platform as infrastructure-as-code: the Azure ML workspace // and its required linked resources (Storage, Key Vault, Application Insights, // Container Registry), a scale-to-zero training compute cluster, and the // least-privilege managed-identity role assignment the workspace needs on its // storage. Deploy the non-prod parameters first, validate, then prod. // // HOW TO RUN (what-if FIRST, always) // az deployment group what-if -g <rg> \ // --template-file infra.bicep \ // --parameters env=nonprod workspaceName=mlw-platform-nonprod // Then deploy: // az deployment group create -g <rg> \ // --template-file infra.bicep \ // --parameters env=nonprod workspaceName=mlw-platform-nonprod // // EXPECTED RESULTS // The workspace 'ML Workspace – Platform – <Stage>' with linked Storage/KeyVault/ // AppInsights/ACR, the training cluster 'Training Cluster – Platform – NonProd' // (min 0 nodes = scale to zero), and a Storage Blob Data Contributor assignment // to the workspace identity — created idempotently. Re-running converges. // // White-label. Deploy plan/what-if first, through the reviewed pipeline. // NOTE: API versions below are marked *verify* — confirm current supported // apiVersion values and feature availability in your tenant/cloud. //

==========

train_register_pipeline.py

Purpose: Submits a reproducible Azure ML pipeline job (prep -> train -> evaluate) using the azure-ai-ml SDK v2 and MLflow tracking, then CONDITIONALLY registers the trained model in the workspace registry with lineage. Registration is a metric gate: the model registers only when --register is passed AND the evaluated metric clears --min-auc. Without --register the script runs a safe DRY-RUN (train + evaluate + print the gate decision, no registration). The accept/reject decision and the threshold value are the DATA-SCIENCE OWNER's — this script only enforces the number they set. HOW TO RUN # 1) Dry-run: train + evaluate, print gate decision, DO NOT register python train_register_pipeline.py --workspace mlw-platform-nonprod \ --resource-group <rg> --data-asset churn-training-data --min-auc 0.80 # 2) Gated register: register a new version ONLY if metric >= threshold python train_register_pipeline.py --workspace mlw-platform-nonprod \ --resource-group <rg> --data-asset churn-training-data --min-auc 0.80 --register Auth uses DefaultAzureCredential (your 'az login' locally; managed identity / OIDC federation in CI/CD — no keys). Requires: azure-ai-ml, mlflow, azureml-mlflow, scikit-learn (pin these in requirements.txt, R1). EXPECTED RESULTS A pipeline job named 'Train-Eval Pipeline – Churn – NonProd' visible under Studio -> Jobs, with parameters/metrics/model logged to MLflow and lineage back to the data asset + environment. With --register and a passing metric, a new 'churn-classifier' version labelled stage=Candidate. NOTES White-label. Registration defaults OFF (dry-run) and is metric-gated when on. *Verify current azure-ai-ml / mlflow versions and the model-promotion mechanism (MLflow classic stages are deprecated; Azure ML uses versions + labels/tags).*

==========

```
from __future__ import annotations
```

How to run: # 1) Dry-run: train + evaluate, print gate decision, DO NOT register python train_register_pipeline.py --workspace mlw-platform-nonprod \ --resource-group <rg> --data-asset churn-training-data --min-auc 0.80 # 2) Gated register: register a new version ONLY if metric >= threshold python train_register_pipeline.py --workspace mlw-platform-nonprod \ --resource-group <rg> --data-asset churn-training-data --min-auc 0.80 --register Auth uses DefaultAzureCredential (your 'az login' locally; managed identity / OIDC federation in CI/CD — no keys). Requires: azure-ai-ml, mlflow, azureml-mlflow, scikit-learn (pin these in requirements.txt, R1). EXPECTED RESULTS A pipeline job named 'Train-Eval Pipeline – Churn – NonProd' visible under Studio -> Jobs, with parameters/metrics/model logged to MLflow and lineage back to the data asset + environment. With --register and a passing metric, a new 'churn-classifier' version labelled stage=Candidate. NOTES White-label. Registration defaults OFF (dry-run) and is metric-gated when on. *Verify current azure-ai-ml / mlflow versions and the model-promotion mechanism (MLflow classic stages are deprecated; Azure ML uses versions + labels/tags).*

==========

```
from __future__ import annotations
```